

Паттерны программирования

Лекция 1
Введение в ООП на
примере языка Ruby

Учебные вопросы

1. Введение в язык
2. Классы и объекты. Переменные и методы
объекта и класса
3. Наследование. Инкапсуляция.
4. Структура встроенных классов языка.
Строки и регулярные выражения

Наследование

```
class Car
```

```
  attr_accessor :odometer  
  attr_accessor :gas_used
```

```
  def mileage  
    @odometer / @gas_used  
  end
```

```
  def accelerate  
    puts "Floor it!"  
  end
```

```
  def sound_horn  
    puts "Beep! Beep!"  
  end
```

```
  def steer ← Копируем!  
    puts "Turn front 2 wheels."  
  end
```

```
end
```

```
class Truck
```

```
  attr_accessor :odometer  
  attr_accessor :gas_used
```

```
  def mileage  
    @odometer / @gas_used  
  end
```

```
  def accelerate  
    puts "Floor it!"  
  end
```

```
  def sound_horn  
    puts "Beep! Beep!"  
  end
```

```
  def steer ← Вставляем!  
    puts "Turn front 2 wheels."  
  end
```

```
end
```

```
class Motorcycle
```

```
  attr_accessor :odometer  
  attr_accessor :gas_used
```

```
  def mileage  
    @odometer / @gas_used  
  end
```

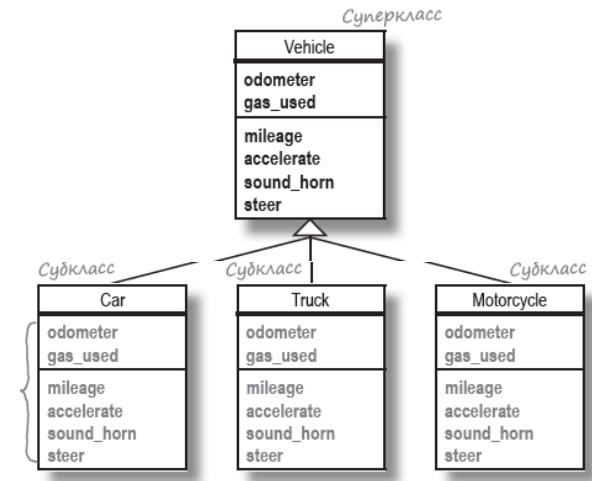
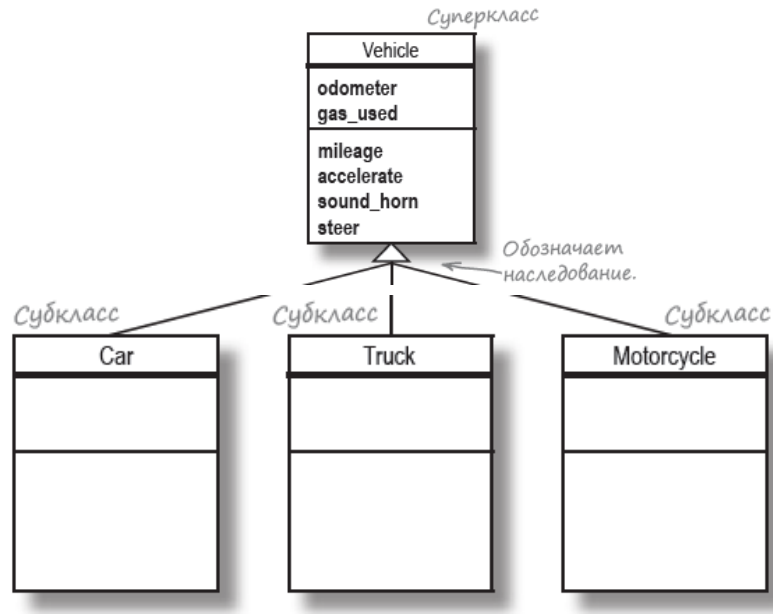
```
  def accelerate  
    puts "Floor it!"  
  end
```

```
  def sound_horn  
    puts "Beep! Beep!"  
  end
```

```
  def steer ← Вставляем!  
    puts "Turn front 2 wheels."  
  end
```

```
end
```

Наследование



Наследование

```
class Car < Vehicle
  //
end
class Truck < Vehicle
  //
end
class Motorcycle < Vehicle
  //
end
```

```
class Car < Vehicle
  //
end
class Truck < Vehicle
  attr_accessor :cargo
  def load_bed(contents)
    puts "Securing #{contents} in the truck bed."
    @cargo = contents
  end
end
class Motorcycle < Vehicle
  def steer
    puts "Turn front wheel."
  end
end
```

Наследование

```
class Person

  def greeting
    puts "Hello!"
  end

end

class Friend < Person

  def greeting
    super
    puts "Glad to see you!"
  end

end
```

```
class Person

  def greeting
    "Hello!"
  end

end

class Friend < Person

  def greeting
    basic_greeting = super
    "#{basic_greeting} Glad to see you!"
  end

end

puts Friend.new.greeting
```

Возвращаемое значение метода.

Переменной basic_greeting присваивается «Hello!»

Hello! Glad to see you!

Наследование

```
class Person
```

```
  def greet_by_name(name)
    "Hello, #{name}!"
  end
```

```
end
```

```
class Friend < Person
```

```
  def greet_by_name(name)
    basic_greeting = super(name)
    "#{basic_greeting} Glad to see you!"
  end
```

```
end
```

```
puts Friend.new.greet_by_name("Meghan")
```

При вызове метода
передается аргумент.

```
class Friend < Person
```

```
  def greet_by_name(name)
    basic_greeting = super
    "#{basic_greeting} Glad to see you!"
  end
```

```
end
```

```
puts Friend.new.greet_by_name("Bert")
```

Метод `greet_by_name`
класса `Friend` должен
вызываться с аргу-
ментом «name»...

...поэтому
аргумент
«name» также
будет передан
методу `greet_`
`by_name` класса
`Person`.

```
Hello, Bert! Glad to see you!
```

Будьте
осторожны!

Вызовы
`super`
и `super()` —
не одно
и то же.

Ключевое слово `super` без ско-
бок вызывает переопределен-
ный метод с теми же аргумен-
тами, которые были переданы
переопределяющему методу.
С другой стороны, вызов
`super()` вызывает переопреде-
ленный метод без аргументов,
даже если переопределяющий
метод получает аргументы.

Наследование

- Метод `to_s` преобразует объект в строку для вывода.
- Метод `inspect` преобразует объект в строку для отладки.
- Метод `class` сообщает, экземпляром какого класса является объект.
- Метод `methods` выводит список методов экземпляра, поддерживаемых объектом.
- Метод `instance_variables` возвращает список переменных экземпляра, содержащихся в объекте.

```
class Student    # повторное открытие класса
  # Предполагается, что в классе Person есть метод say_hello...
  alias :say_hi :say_hello

  def say_hello
    puts "Привет."
  end

  def formal_greeting
    # Поприветствовать так, как принято в суперклассе.
    say_hi
  end
end
```

Наследование

equal?

==

eql?

hash

```
flag1 = (1 == 1.0)      # true
flag2 = (1.eql?(1.0))   # false
```

```
case an_object
  when String
    puts "Это строка."
  when Numeric
    puts "Это число."
  else
    puts "Это что-то совсем другое."
end
```

Эта конструкция работает, потому что в классе `Module` реализован метод `===`, проверяющий, что параметр принадлежит тому же классу, что вызывающий объект (или одному из его предков). Поэтому, если `an_object` – это строка «cat», то выражение `String === an_object` окажется истинным, и будет выбрана первая ветвь в предложении `case`.

Наконец, в Ruby реализован оператор сопоставления с образцом `=~`. Традиционно он применяется для сопоставления строки с регулярным выражением. Но если вы найдете ему применение в других классах, то можете переопределить.

Наследование

```
class Bank
  def open_safe
    # ...
  end

  def close_safe
    # ...
  end

  private :open_safe, :close_safe

  def make_withdrawal(amount)
    if access_allowed
      open_safe
      get_cash(amount)
      close_safe
    end
  end

  # остальные методы закрытые

  private

  def get_cash
    # ...
  end

  def access_allowed
    # ...
  end
end
```

```
class Person
  attr_reader :name, :age, :pay_scale
  protected :age
  private :pay_scale

  def initialize(name, age, pay_scale)
    @name, @age, @pay_scale = name, age, pay_scale
  end

  def <=>(other)
    age <=> other.age # allowed by protected
  end

  def same_rank?(other)
    pay_scale == other.pay_scale # not allowed by private
  end

  def rank
    case pay_scale
    when 1..3
      "lower"
    when 4..6
      "middle"
    when 7..9
      "high"
    end
  end
end
```

```
s1 = "cat"
```

```
def s1.upcase
  "CaT"
end
```

```
s1_dup = s1.dup
s1_clone = s1.clone

s1                                     #=> "cat"
s1_dup.upcase                         #=> "CaT"
s1_clone.upcase                       #=> "CaT"
```

Наследование

```
class Bank
  def open_safe
    # ...
  end

  def close_safe
    # ...
  end

  private :open_safe, :close_safe

  def make_withdrawal(amount)
    if access_allowed
      open_safe
      get_cash(amount)
      close_safe
    end
  end

  # остальные методы закрытые

  private

  def get_cash
    # ...
  end

  def access_allowed
    # ...
  end
end
```

```
class Person
  attr_reader :name, :age, :pay_scale
  protected :age
  private :pay_scale

  def initialize(name, age, pay_scale)
    @name, @age, @pay_scale = name, age, pay_scale
  end

  def <=>(other)
    age <=> other.age # allowed by protected
  end

  def same_rank?(other)
    pay_scale == other.pay_scale # not allowed by private
  end

  def rank
    case pay_scale
    when 1..3
      "lower"
    when 4..6
      "middle"
    when 7..9
      "high"
    end
  end
end
```

```
s1 = "cat"
```

```
def s1.upcase
  "CaT"
end
```

```
s1_dup = s1.dup
s1_clone = s1.clone

s1                                     #=> "cat"
s1_dup.upcase                         #=> "CaT"
s1_clone.upcase                       #=> "CaT"
```

Структура встроенных классов языка

```
1.class      # => Fixnum: число 1 относится к классу Fixnum
0.0.class    # => Float: числа с плавающей точкой относятся к классу Float
true.class   # => TrueClass: true – единственный экземпляр класса TrueClass
false.class  # => FalseClass
nil.class    # => NilClass
```

Структура встроенных классов языка

<code>__LINE__</code>	<code>case</code>	<code>ensure</code>	<code>not</code>	<code>then</code>
<code>__ENCODING__</code>	<code>class</code>	<code>false</code>	<code>or</code>	<code>true</code>
<code>__FILE__</code>	<code>def</code>	<code>for</code>	<code>redo</code>	<code>undef</code>
<code>BEGIN</code>	<code>defined?</code>	<code>if</code>	<code>rescue</code>	<code>unless</code>
<code>END</code>	<code>do</code>	<code>in</code>	<code>retry</code>	<code>until</code>
<code>alias</code>	<code>else</code>	<code>module</code>	<code>return</code>	<code>when</code>
<code>and</code>	<code>elsif</code>	<code>next</code>	<code>self</code>	<code>while</code>
<code>begin</code>	<code>end</code>	<code>nil</code>	<code>super</code>	<code>yield</code>
<code>break</code>				

Структура встроенных классов языка

```
minimum = if x < y then x else y end
```

**В руби ВСЕ -
ВЫРАЖЕНИЯ**

Структура встроенных классов языка

```
Терминал - ars@ars-Lenovo-IdeaPad-S145-15AST: ~/repo/36_ruby_labs/RR/10
Файл Правка Вид Терминал Вкладки Справка

irb(main):006:0> 1.class.superclass.superclass.superclass.name
=> "BasicObject"
irb(main):007:0> 1.class.superclass.superclass.superclass.methods
=> [:new, :allocate, :superclass, :<=>, :<=, :>=, :==, :===, :included_modules,
:include?, :ancestors, :attr, :attr_reader, :attr_writer, :attr_accessor, :freeze,
:inspect, :public_instance_methods, :instance_methods, :const_missing, :protected_instance_methods, :private_instance_methods, :const_set, :constants, :remove_class_variable, :class_variable_get, :class_variable_set, :class_variable_defined?, :const_get, :const_defined?, :<, :>, :public_constant, :class_variables, :private_constant, :deprecate_constant, :singleton_class?, :const_source_location, :to_s, :class_eval, :include, :module_exec, :module_eval, :prepend, :undef_method, :alias_method, :class_exec, :remove_method, :method_defined?, :name, :private_class_method, :public_method_defined?, :private_method_defined?, :protected_method_defined?, :public_class_method, :instance_method, :public_instance_method, :define_method, :autoload, :autoload?, :dup, :itself, :yield_self, :then, :taint, :tainted?, :untaint, :untrust, :untrusted?, :trust, :frozen?, :methods, :singleton_methods, :protected_methods, :private_methods, :public_methods, :instance_variables, :instance_variable_get, :instance_variable_set, :instance_variable_defined?, :remove_instance_variable, :instance_of?, :kind_of?, :is_a?, :tap, :class, :singleton_class, :display, :clone, :hash, :public_send, :method, :public_method, :singleton_method, :define_singleton_method, :extend, :to_enum, :enum_for, :=~, :!~, :nil?, :eql?, :respond_to?, :object_id, :send, :__send__, :!, :!=, :equal?, :_id_, :instance_eval, :instance_exec]
irb(main):008:0> 
```

Структура встроенных классов языка

<https://ruby-doc.org/core-2.7.1/BasicObject.html>

Структура встроенных классов языка

```
Терминал - ars@ars-Lenovo-IdeaPad-S145-15AST: ~/repo/36_ruby_labs/RR/10
Файл  Правка  Вид  Терминал  Вкладки  Справка

irb(main):008:0> 1.class.superclass.superclass.name
=> "Object"
irb(main):009:0> 1.class.superclass.superclass.methods
=> [:new, :allocate, :superclass, :<=>, :<=, :>=, :==, :===, :included_modules,
:include?, :ancestors, :attr, :attr_reader, :attr_writer, :attr_accessor, :freeze,
:inspect, :public_instance_methods, :instance_methods, :const_missing, :protected_instance_methods, :private_instance_methods, :const_set, :constants, :remove_class_variable, :class_variable_get, :class_variable_set, :class_variable_defined?, :const_get, :const_defined?, :<, :>, :public_constant, :class_variables, :private_constant, :deprecate_constant, :singleton_class?, :const_source_location, :to_s, :class_eval, :include, :module_exec, :module_eval, :prepend, :undef_method, :alias_method, :class_exec, :remove_method, :method_defined?, :name, :private_class_method, :public_method_defined?, :private_method_defined?, :protected_method_defined?, :public_class_method, :instance_method, :public_instance_method, :define_method, :autoload, :autoload?, :dup, :itself, :yield_self, :then, :taint, :tainted?, :untaint, :untrust, :untrusted?, :trust, :frozen?, :methods, :singleton_methods, :protected_methods, :private_methods, :public_methods, :instance_variables, :instance_variable_get, :instance_variable_set, :instance_variable_defined?, :remove_instance_variable, :instance_of?, :kind_of?, :is_a?, :tap, :class, :singleton_class, :display, :clone, :hash, :public_send, :method, :public_method, :singleton_method, :define_singleton_method, :extend, :to_enum, :enum_for, :~, :!~, :nil?, :eql?, :respond_to?, :object_id, :send, :__send__, :!, :!=, :equal?, :_id_, :instance_eval, :instance_exec]
irb(main):010:0> 
```

Встроенные структуры данных

```
class Book
  attr_reader :author, :title

  def initialize(author, title)
    @author = author
    @title = title
  end

  def ==(other)
    self.class === other and
    other.author == @author and
    other.title == @title
  end

  alias eql? ==

  def hash
    @author.hash ^ @title.hash # XOR
  end
end

book1 = Book.new 'matz', 'Ruby in a Nutshell'
book2 = Book.new 'matz', 'Ruby in a Nutshell'

reviews = {}

reviews[book1] = 'Great reference!'
reviews[book2] = 'Nice and compact!'

reviews.length #=> 1
```

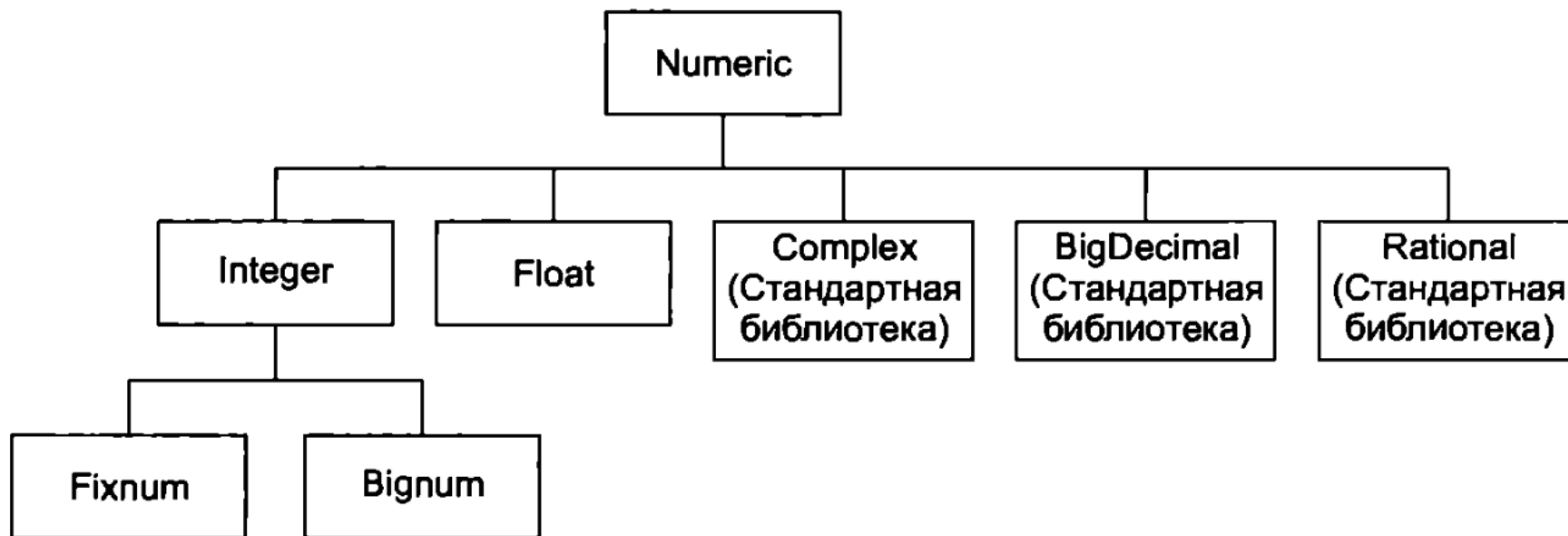
Структура встроенных классов языка



```
Терминал - ars@ars-Lenovo-IdeaPad-S145-15AST: ~/repo/36_ruby_labs/RR/10
Файл  Правка  Вид  Терминал  Вкладки  Справка

:include?, :ancestors, :attr, :attr_reader, :attr_writer, :attr_accessor, :freeze, :inspect, :public_instance_methods, :instance_methods, :const_missing, :protected_instance_methods, :private_instance_methods, :const_set, :constants, :remove_class_variable, :class_variable_get, :class_variable_set, :class_variable_defined?, :const_get, :const_defined?, :<, :>, :public_constant, :class_variables, :private_constant, :deprecate_constant, :singleton_class?, :const_source_location, :to_s, :class_eval, :include, :module_exec, :module_eval, :prepend, :undef_method, :alias_method, :class_exec, :remove_method, :method_defined?, :name, :private_class_method, :public_method_defined?, :private_method_defined?, :protected_method_defined?, :public_class_method, :instance_method, :public_instance_method, :define_method, :autoload, :autoload?, :dup, :itself, :yield_self, :then, :taint, :tainted?, :untaint, :untrust, :untrusted?, :trust, :frozen?, :methods, :singleton_methods, :protected_methods, :private_methods, :public_methods, :instance_variables, :instance_variable_get, :instance_variable_set, :instance_variable_defined?, :remove_instance_variable, :instance_of?, :kind_of?, :is_a?, :tap, :class, :singleton_class, :display, :clone, :hash, :public_send, :method, :public_method, :singleton_method, :define_singleton_method, :extend, :to_enum, :enum_for, :==, :!=, :nil?, :eql?, :respond_to?, :object_id, :send, :__send__, :!, :!=, :equal?, :_id_, :instance_eval, :instance_exec]
irb(main):010:0> 1.class.superclass.name
=> "Numeric"
irb(main):011:0> 1.class.name
=> "Integer"
irb(main):012:0> 
```

Структура встроенных классов языка



Структура встроенных классов языка

```
1048576      # число в обычной записи  
1_048_576    # то же самое значение
```

```
0b10010110   # двоичное  
0b1211        # ошибка!  
01234         # восьмеричное (основание 8)  
01823         # ошибка!  
0xdeadbeef    # шестнадцатеричное (основание 16)  
0xDEADBEEF    # то же самое  
0xdeadpork    # ошибка!
```

```
3.14          # число пи с двумя знаками  
-0.628        #  $-2\pi$ , поделенное на 10, с тремя знаками  
6.02e23       # число Авогадро  
6.626068e-34  # постоянная Планка
```

Структура встроенных классов языка

```
Терминал - ars@ars-Lenovo-IdeaPad-S145-15AST: ~/repo/36_ruby_labs/RR/10
Файл  Правка  Вид  Терминал  Вкладки  Справка

irb(main):011:0> 1.class.name
=> "Integer"
irb(main):012:0> 1.class.methods
=> [:sqrt, :allocate, :superclass, :<=>, :<=, :>=, :==, :===, :included_modules,
   :include?, :ancestors, :attr, :attr_reader, :attr_writer, :attr_accessor, :freeze, :inspect, :public_instance_methods, :instance_methods, :const_missing, :protected_instance_methods, :private_instance_methods, :const_set, :constants, :remove_class_variable, :class_variable_get, :class_variable_set, :class_variable_defined?, :const_get, :const_defined?, :<, :>, :public_constant, :class_variables, :private_constant, :deprecate_constant, :singleton_class?, :const_source_location, :to_s, :class_eval, :include, :module_exec, :module_eval, :prepend, :undef_method, :alias_method, :class_exec, :remove_method, :method_defined?, :name, :private_class_method, :public_method_defined?, :private_method_defined?, :protected_method_defined?, :public_class_method, :instance_method, :public_instance_method, :define_method, :autoload, :autoload?, :dup, :itself, :yield_self, :then, :taint, :tainted?, :untaint, :untrust, :untrusted?, :trust, :frozen?, :methods, :singleton_methods, :protected_methods, :private_methods, :public_methods, :instance_variables, :instance_variable_get, :instance_variable_set, :instance_variable_defined?, :remove_instance_variable, :instance_of?, :kind_of?, :is_a?, :tap, :class, :singleton_class, :display, :clone, :hash, :public_send, :method, :public_method, :singleton_method, :define_singleton_method, :extend, :to_enum, :enum_for, :~, :!~, :nil?, :eql?, :respond_to?, :object_id, :send, :__send__, :!, :!=, :equal?, :__id__, :instance_eval, :instance_exec]
irb(main):013:0> 
```

Структура встроенных классов языка

```
Float::MIN      # 2.2250738585072e-308 (на конкретной машине)  
Float::MAX      # 1.79769313486232e+308  
Float::EPSILON  # 2.22044604925031e-16
```

```
    a = 64**2      # 4096  
    b = 64**0.5    # 8.0  
    c = 64**0      # 1  
    d = 64**-1     # 0.015625
```

```
    3 / 4.0        # 0.75  
    3.0 / 4.0      # 0.75
```

```
    z = x.to_f / y  
    z = Float(x) / y
```

```
    pi = 3.14159  
    new_pi = pi.round # 3  
    temp = -47.6  
    temp2 = temp.round # -48
```

Структура встроенных классов языка

```
pi = 3.1415926535
pi6 = eval(sprintf("%8.6f",pi)) # 3.141593
pi5 = eval(sprintf("%8.5f",pi)) # 3.14159
pi4 = eval(sprintf("%8.4f",pi)) # 3.1416
```

```
x = 1000001.0/0.003
y = 0.003*x
if y == 1000001.0
    puts "да"
else
    puts "нет"
end
```

```
class Float

    EPSILON = 1e-6 # 0.000001

    def ==(x)
        (self-x).abs < EPSILON
    end

end
```

```
x = 1000001.0/0.003
y = 0.003*x
```

Структура встроенных классов языка

```
x = 345.6789
i = 123
printf("x = %6.2f\n", x)    # x = 345.68
printf("x = %9.2e\n", x)    # x = 3.457e+02
printf("i = %5d\n", i)      # i = 123
printf("i = %05d\n", i)     # i = 00123
printf("i = %-5d\n", i)     # i = 123
```

Чтобы сохранить результат в строке, а не печатать его немедленно, воспользуйтесь методом `sprintf`. При следующем обращении возвращается строка:

```
str = sprintf("%5.1f", x)    # "345.7"
```

* Порядок вызова: 'формат % значение'

```
str = "%5.1f" % x           # "345.7"
```

```
str = "%6.2f, %05d" % (x, i) # "345.68, 00123"
```

Структура встроенных классов языка

```
require 'bigdecimal'

x = BigDecimal("3.2")
y = BigDecimal("2.0")
z = BigDecimal("1.2")

if (x - y) == z
  puts "равны" # печатается "равны"!
else
  puts "не равны"
end

a = x*y*z
a.to_s      # "0.768E1" (по умолчанию: научная нотация)
a.to_s("F") # "7.68"   (обычная запись)
```

Структура встроенных классов языка

Следующие подбиблиотеки являются дополнениями к `BigDecimal`.

- `bigdecimal/math` – модуль `BigMath`;
- `bigdecimal/jacobian` – методы для вычисления матрицы Якоби;
- `bigdecimal/ludcmp` – модуль `LUSolve`, разложение матрицы в произведение верхнетреугольной и нижнетреугольной;
- `bigdecimal/newton` – методы `nlolve` и `norm`.

```
r = Rational(1,2)  # 1/2 или 0.5
s = Rational(1,3)  # 1/3 или 0.3333...
t = Rational(1,7)  # 1/7 или 0.14...
u = Rational(6,2)  # "то же самое, что" 3.0
z = Rational(1,0)  # ошибка!
```

Структура встроенных классов языка

```
z+t      # Rational(9, 14)
r-t      # Rational(5, 14)
r*s      # Rational(1, 6)
r/s      # Rational(3, 2)
```

```
x = Rational(1000001,1)/Rational(3,1000)
y = Rational(3,1000)*x
if y == 1000001.0
  puts "да"          # Теперь получаем "да"!
else
  puts "нет"
end
```

Конечно, не любая операция дает рациональное же число в качестве результата:

```
x = Rational(9,16)      # Rational(9, 16)
Math.sqrt(x)            # 0.75
x**0.5                  # 0.75
x**Rational(1,2)        # 0.75
```

Структура встроенных классов языка

```
z = Complex(3,5)      # 3+5i  
a = 3.im              # 3i  
b = 5 - 2.im          # 5-2i  
z = Complex.polar(5,Math::PI/2.0)  # радиус, угол
```

Mathn

Структура встроенных классов языка

```
t0 = Time.new
```

Синонимом служит Time.now:

```
t0 = Time.now
```

```
t1 = Time.mktime(2014)           # 1 января 2014 в 0:00:00
t2 = Time.mktime(2014,3)
t3 = Time.mktime(2014,3,15)
t4 = Time.mktime(2014,3,15,21)
t5 = Time.mktime(2014,3,15,21,30)
t6 = Time.mktime(2014,3,15,21,30,15) # 15 марта 2014 21:30:15
```

Структура встроенных классов языка

- Класс `Time` – преимущественно обертывает соответствующие функции из стандартной библиотеки языка C. Они как правило опираются на точку отсчета в UNIX и потому не способны представлять моменты времени раньше 1970 года.
- Класс `Date` создан для преодоления недостатков класса `Time`. Он без труда справляется с датами в более отдаленном прошлом, например, позволяет представить день рождения Леонардо да Винчи (15 апреля 1452), и знает о реформе календаря. Но у него есть свои слабые места: он работает только с датами, игнорируя время.
- Класс `DateTime` наследует `Date` и пытается компенсировать отсутствующие в нем возможности. Он может представлять даты не хуже `Date` и время не хуже `Time`. Часто его способ представления даты и времени оказывается «наилучшим».

Структура встроенных классов языка

%b Сокращенное название месяца ("Jan")
%B Полное название месяца ("January")
%c Предпочтительное представление локальной даты и времени
%d День месяца (1..31)
%j Порядковый номер дня в году (1..366); так называемая "юлианская дата"
%m Номер месяца (1..12)
%w Номер дня недели (0..6)
%x Предпочтительное представление даты без времени
%y Двухзначный год (без указания века)
%Y Четырехзначный год
%Z Название часового пояса
%% Знак "%"

<https://ruby-doc.org/core-2.7.1/Time.html>

Строки

```
s1 = 'Это строка'          # Это строка
s2 = 'Г-жа О\'Лири'        # Г-жа О'Лири
s3 = 'Смотри в C:\\TEMP'    # Смотри в C:\TEMP
```

```
s1 = "Это знак табуляции: (\t)"
s2 = "Несколько символов забоя: xyz\b\b\b"
s3 = "Это тоже знак табуляции: \011"
s4 = "А это символы подачи звукового сигнала: \a \007"
s5 = "Это символ снеговика в unicode: \u2603"
```

```
puts "#{temp_f} по Фаренгейту равно #{temp_c} по Цельсию"
puts "Значение определителя равно #{b*b - 4*a*c}."
puts "#{word} это #{word.reverse} наоборот."
```

```
str = "Ответ равен #{ def factorial(n)
  n==0 ? 1 : n*factorial(n-1)
end
answer = factorial(3) * 7}, естественно."
# Ответ равен 42, естественно.
```

```
puts "$gvar = #{$gvar} и ivar = #@ivar."
```

```
str = proc do |name, nation|
  "#{name} - мое имя, а #{nation} - моя страна."
end
s2 = str.call("Гулливер Фойл", "Терра")
= Гулливер Фойл - мое имя, а Терра - моя страна.
```

Строки

The image shows a Ruby script in a Notepad++ window and its execution output in a Command Prompt window.

Script (strings.rb):

```
1 s1 = %q[What a fuck "yeah"]
2 s2 = %q[ (\t) dfdfdf ]
3 s3 = %Q[ (\t) dfdfdf ]
4
5 puts s1, s2, s3
6
7 str = "Once upon a time \n ther was a hole \n the end\n"
8
9
10 str.each_line do |line|
11   puts "The length is #{line.length}"
12 end
13
14 str.each_line.with_index do |line, index|
15   puts "The length of #{index} string is #{line.length}"
16 end
17 count = 0
18 str.each_byte do
19   |byte|
20   print "#{byte}, "
21   count += 1
22 end
23 puts "\n#{count}"
24
25 str.each_char {|char| print "#{char} - #{char.to_i}\t"}
26
27 puts
28 puts str.codepoints.to_a.length
29 str.codepoints.to_a.each {|char| print "#{char}\t"}
30
31 str = "Пашный ТЕКСТ 你~你好世界"
32 puts str.codepoints.to_a.length
33 str.codepoints.to_a.each {|char| print "#{char}\t"}
34
35
```

Command Prompt Output:

```
C:\Sites\chap2>ruby strings.rb
What a fuck "yeah"
( \t) dfdfdf
( \t) dfdfdf
The length is 18
The length is 18
The length is 9
The length of 0 string is 18
The length of 1 string is 9
The length of 2 string is 9
15, 32, 97, 32, 104, 111, 108, 101, 32, 10, 32, 116, 104, 101, 114, 32, 119, 97, 1
45
0 - 0 n - 0 c - 0 e - 0   - 0 u - 0 p - 0 o - 0 n - 0   - 0 a - 0   - 0 t - 0 i - 0 m - 0
   e - 0   - 0
   - 0   - 0 t - 0   h - 0 e - 0 r - 0   - 0 w - 0 a - 0 s - 0   - 0 a - 0   - 0 h - 0 o - 0
   1 - 0   e - 0   - 0
   - 0   - 0 t - 0   h - 0 e - 0   - 0 e - 0 n - 0 d - 0
   - 0
45
79      110  99   101  32   117   112   111   110  32   97   32   116   105   109   101  32   10,  32   116   104   101   114   32   119   97,  1
      32   104   111   108   101  32   10   32   116   104   101  32   101   110   100,  10,
      100   10   20
1056  1072  1079  1085  1099  1081  32   1058  1045  1050  1057  1058  32   1025  96
      126  20320  22909  19990  30028
C:\Sites\chap2>
```

Строки

```
s1 = %q[Как сказал Магритт, "Ceci n'est pas une pipe."]
s2 = %q[Это не табуляция: (\t)]      # то же, что: 'Это не табуляция: \t'
s3 = %Q[A это табуляция: (\t)]      # то же, что: "A это табуляция: \t"
```

```
s1 = %q(Билл сказал: "Боб сказал: 'This is a string.'")
s2 = %q{Другая строка.}
s3 = %q<В этой строке есть специальные символы '"[](){}>.
```

```
str1 = "Карл"
x = str1.length # 4
str2 = "Дойль"
x = str2.size # 5
```

```
s1 = "Была темная грозовая ночь."
words = s1.split      # ["Была", "темная", "грозовая", "ночь"]
```

```
s2 = "яблоки, груши, персики"
list = s2.split(", ") # ["яблоки", "груши", "персики"]
```

```
s3 = "львы и тигры и медведи"
zoo = s3.split(/ and /) # ["львы", "тигры", "медведи"]
```

```
str = "alpha,beta,gamma,, "
list1 = str.split(",")      # ["alpha", "beta", "gamma"]
list2 = str.split(",", 2)   # ["alpha", "beta,gamma,, "]
list3 = str.split(",", 4)   # ["alpha", "beta", "gamma", ", "]
list4 = str.split(",", 8)   # ["alpha", "beta", "gamma", "", ""]
list5 = str.split(",", -1)  # ["alpha", "beta", "gamma", "", ""]
```

```
str = "I am a leaf on the wind..."
```

```
# Строка интерпретируется буквально, а не как регулярное выражение
arr = str.scan("a") # ["a", "a", "a"]
```

```
# При сопоставлении с регулярным выражением возвращаются все соответствия
arr = str.scan(/\w+/)
# ["I", "am", "a", "leaf", "on", "the", "wind"]
```

```
# Блоку будут по одному передаваться найденные соответствия
str.scan(/\w+/) {|x| puts x }
```

Строки

```
require 'strscan'
str = "Смотри как я парю!"
ss = StringScanner.new(str)
loop do
  word = ss.scan(/\w+/) # Получать по одному слову
  break if word.nil?
  puts word
  sep = ss.scan(/\W+/) # Получить следующий фрагмент, не являющийся словом
  break if sep.nil?
end
```

```
name = "Боб"
age = 28
str = sprintf("Привет, %s... Похоже, тебе %d лет.", name, age)
```

```
str = sprintf("%-20s %3d", name, age)
```

```
str = "%-20s %3d" % [name, age] # То же, что и выше
```

```
str = "Моби Дик"
```

```
s1 = str.ljust(12)
```

```
s2 = str.center(12)
```

```
s3 = str.rjust(12)
```

```
str = "Капитан Ахав"
```

```
s1 = str.ljust(20, "+")
```

```
s2 = str.center(20, "-")
```

```
s3 = str.rjust(20, "123")
```

```
s1 = "Бостонское чаепитие"
```

```
s2 = s1.downcase
```

```
s3 = s2.upcase
```

```
# "Моби Дик"
```

```
# "  Моби Дик  "
```

```
# "      Моби Дик"
```

```
# "Капитан Ахав+++++++"
```

```
# "—Капитан Ахав—"
```

```
# "12312312Капитан Ахав"
```

```
# "бостонское чаепитие"
```

```
# "БОСТОНСКОЕ ЧАЕПИТИЕ"
```

Строки

```
s1 = "Бостонское чаепитие"
s2 = s1.downcase           # "бостонское чаепитие"
s3 = s2.upcase             # "БОСТОНСКОЕ ЧАЕПИТИЕ"

s4 = s1.capitalize        # "Бостонское чаепитие"
s5 = s2.capitalize        # "Бостонское чаепитие"
s6 = s3.capitalize        # "Бостонское чаепитие"

n1 = "abc".casecmp("xyz")  # -1
n2 = "abc".casecmp("XYZ")  # -1
n3 = "ABC".casecmp("xyz")  # -1
n4 = "ABC".casecmp("abc")  # 0
n5 = "xyz".casecmp("abc")  # 1
```

upcase!, downcase!, capitalize!, swapcase!

```
if string =~ /[a-z]/
  puts "строка содержит символы из нижнего регистра"
end

if string =~ /[A-Z]/
  puts "строка содержит символы из верхнего регистра"
end

if string =~ /[A-Z]/ and string =~ /a-z/
  puts "строка содержит символы из разных регистров"
end
```

Строки

```
str = "Шалтай-Болтай"
sub1 = str[7,4]          # "Болт"
sub2 = str[7,99]         # "Болтай" (выход за границу строки допускается)
sub3 = str[10,-4]        # nil (отрицательная длина)
```

```
str1 = "Алиса"
sub1 = str1[-3,3]        # "иса"
str2 = "В Зазеркалье"
sub3 = str2[-8,6]        # "зеркал"
```

```
str = "Уинстон Черчилль"
sub1 = str[8..13]        # "Черчил"
sub2 = str[-4..-1]       # "илль"
sub3 = str[-1..-4]       # nil
sub4 = str[25..30]       # nil
```

```
str = "Alistair Cooke"
sub1 = str[/l..t/]       # "list"
sub2 = str[/s.*r/]       # "stair"
sub3 = str[/foo/]        # nil
```

```
str = "theater"
sub1 = str["heat"]       # "heat"
sub2 = str["eat"]        # "eat"
sub3 = str["ate"]        # "ate"
sub4 = str["beat"]       # nil
sub5 = str["cheat"]      # nil
```

Строки

```
str1 = "Шалтай-Болтай"
str1[7,3] = "Хва"          # "Шалтай-Хватай"

str2 = "Алиса"
str2[-3,3] = "ександра"    # "Александра"

str3 = "В Зазеркалье"
str3[-9,9] = "стеколье"    # "В Застеколье"

str4 = "Уинстон Черчилль"
str4[8..11] = "Х"          # "Уинстон Хилль"

str5 = "Alistair Cooke"
str5[/e$/] = "ie Monster"  # "Alistair Cookie Monster"

str6 = "theater"
str6["er"] = "re"          # "theatre"

str7 = "Aaron Burr"
str7[0] = 66                # "Baron Burr"
```

```
s1 = "spam, spam, and eggs"
s2 = s1.sub(/spam/, "bacon")      # "bacon, spam, and eggs"

s3 = s2.sub(/(\w+), (\w+)/, '\2, \1,') # "spam, bacon, and eggs"

s4 = "Don't forget the spam."
s5 = s4.sub(/spam/) { |m| m.reverse } # "Don't forget the maps."

s4.sub!(/spam/) { |m| m.reverse }
# s4 теперь равно "Don't forget the maps."

s5 = "alfalfa abracadabra"
s6 = s5.gsub(/a[bl]/, "xx")      # "xxfxxfa xxracadxxra"
s5.gsub!(/[lfdbr]/) { |m| m.upcase + "-" }
# s5 теперь равно "aL-F-aL-F-a aB-R-acaD-aB-R-a"
```

Строки

```
str = "Albert Einstein"
pos1 = str.index(?E)           # 7
pos2 = str.index("bert")      # 2
pos3 = str.index(/in/)        # 8
pos4 = str.index(?W)          # nil
pos5 = str.index("bart")      # nil
pos6 = str.index(/wein/)      # nil

str1 = "mathematics"
flag1 = str1.include? ?e      # true
flag2 = str1.include? "math"  # true
str2 = "Daylight Saving Time"
flag3 = str2.include? ?s      # false
flag4 = str2.include? "Savings" # false

str = "Albert Einstein"
pos1 = str.rindex(?E)         # 7
pos2 = str.rindex("bert")     # 2
pos3 = str.rindex(/in/)       # 13 (найдено самое правое соответствие)
pos4 = str.rindex(?W)         # nil
pos5 = str.rindex("bart")     # nil

str1 = "abracadabra"
sub1 = str1.scan(/a./)
# sub1 теперь равно ["ab","ac","ad","ab"]
str2 = "Acapulco, Mexico"
sub2 = str2.scan(/(.) (c.)/)
# sub2 теперь равно [ ["A","ca"], ["l","co"], ["i","co"] ]
```

Строки

```
class Helium
  def to_s
    "He"
  end
  def to_str
    "гелий"
  end
end

e = Helium.new
print "Элемент "
puts e      # Элемент He
puts "Element is " + e      # Элемент гелий
puts "Element is #{e}"      # Элемент He
```

```
str = "A"
str << [1,2,3].to_s << " " << (3.14).to_s
# str теперь равно "A123 3.14"
```

```
str = gets.chomp      # Прочитать, удалить символ новой строки
s2 = "Some string\n"  # "Some string" (нет символа новой строки)
s3 = s2.chomp!        # s2 теперь тоже равно "Some string"
s4 = "Other string\r\n"
s4.chomp!             # "Other string" (нет символа новой строки)
```

```
str = "abcxyz"
s1 = str.chomp        # "abcxy"
```

Строки

```
str = "abcxyz"
str2 = "123\n"
str3 = "123\r"
str4 = "123\r\n"
s1 = str.chomp           # "abcxyz"
s2 = str2.chomp          # "123"
# Если установлен стандартный разделитель записей, то удаляется не только
# \n, но также \r и \r\n
s3 = str3.chomp          # "123"
s4 = str4.chomp          # "123"
```

```
str1 = "abcxyz"
str2 = "abcxyz"
s1 = str1.chomp("yz")    # "abcx"
s2 = str2.chomp("x")     # "abcxyz"
```

```
etc = "Etc. " * 3        # "Etc. Etc. Etc. "
ruler = "+" + ( "." * 4 + "5" + "." * 4 + "+" ) * 3
# "+....5....+....5....+....5....+"
```

```
x = "123".to_i           # 123
y = Integer("123")       # 123
```

```
x = "junk".to_i          # молча возвращает 0
y = Integer("junk")      # ошибка
```

```
x = "123junk".to_i      # 123
y = Integer("123junk")  # error
```

Строки

```
x = " 123 ".to_i          # 123   coded = "hfCghHIE5LAM."
y = Integer(" 123 ")      # 123

x = "3.1416".to_f         # 3.1416
y = Float("2.718")        # 2.718

x = Float("6.02e23")      # 6.02e23
y = "2.9979246e5".to_f   # 299792.46

puts "Говори, друг, и жми Enter!"

print "Пароль: "
password = gets.chomp

if password.crypt("hf") == coded
  puts "Добро пожаловать!"
else
  puts "Кто ты, орк?"
end
```

Строки

```
require 'zlib'
include Zlib

long_string = ("abcde"*71 + "defghi"*79 + "ghijkl"*113)*371
# long_string состоит из 559097 символов

s1 = Deflate.deflate(long_string, BEST_SPEED)      # 4188 символов
s2 = Deflate.deflate(long_string)                  # 3568 символов
s3 = Deflate.deflate(long_string, BEST_COMPRESSION) # 2120 символов
```

```
s1 = "abracadabra"
a = s1.count("c")      # 1
b = s1.count("bdr")    # 5
```

```
c = s1.count("^a")      # 6
d = s1.count("^bdr")    # 6
```

Дефис обозначает диапазон символов:

```
e = s1.count("a-d")      # 9
f = s1.count("^a-d")     # 2
```

```
s1 = "Star Trek"
s2 = s1.reverse           # "kerT ratS"
s1.reverse!              # s1 теперь равно "kerT ratS"
```

```
phrase = "Now here's a sentence"
phrase.split(" ").reverse.join(" ")
# "sentence a here's Now"
```

Строки

```
s1 = "bookkeeper"
s2 = s1.squeeze      # "bokeper"
s3 = "Hello..."
s4 = s3.squeeze      # "Helo."
```

```
s5 = s3.squeeze(".") # "Hello."
```

```
s1 = "To be, or not to be"
s2 = s1.delete("b")    # "To e, or not to e"
s3 = "Veni, vidi, vici!"
s4 = s3.delete(",!")   # "Veni vidi vici"
```

```
droid = "R2D2"
improved = droid.succ      # "R2D3"
pill = "Vitamin B"
pill2 = pill.succ          # "Vitamin C"
```

```
require 'zlib'
include Zlib
crc = crc32("Hello") # 4157704578
crc = crc32(" world!",crc) # 461707669
crc = crc32("Hello world!") # 461707669 (то же, что и выше)
```

Файлы. Введение

```
file1 = File.new("one")      # Открыть для чтения  
file2 = File.new("two", "w") # Открыть для записи
```

```
file = File.new("three", 0755, File::CREAT|File::WRONLY)
```

```
out = File.new("captains.log", "w")  
# Обработка файла...  
out.close
```

```
trans = File.open("transactions", "w")
```

```
File.open("somefile", "w") do |file|  
  file.puts "Строка 1"  
  file.puts "Строка 2"  
  file.puts "Третья и последняя строка"  
end  
# Теперь файл закрыт
```

```
f1 = File.new("file1", "r+")  
# Чтение/запись, от начала файла.
```

```
f2 = File.new("file2", "w+")  
# Чтение/запись; усесть существующий файл или создать новый.
```

```
f3 = File.new("file3", "a+")  
# Чтение/запись; перейти в конец существующего файла или создать новый.
```

Регулярные выражения

Регулярное выражение	Пояснение
/Ruby/	Соответствует одному слову <i>Ruby</i>
/[Rr]uby/	Соответствует <i>Ruby</i> или <i>ruby</i>
/^abc/	Соответствует <i>abc</i> в начале строки
%r(xyz\$)	Соответствует <i>xyz</i> в конце строки
%r [0-9]*	Соответствует любой последовательности из нуля или более цифр

Модификатор	Назначение
I	Игнорировать регистр
O	Выполнять подстановку выражения только один раз
M	Многострочный режим (точка сопоставляется с символом новой строки)
X	Обобщенное регулярное выражение (допускаются пробелы и комментарии)

Обозначение	Пояснение
^	Начало строки текста (line) или строки символов (string)
\$	Конец строки текста или строки символов
.	Любой символ, кроме символа новой строки (если не установлен многострочный режим)
\w	Символ-часть слова (цифра, буква или подчеркик)
\W	Символ, не являющийся частью слова
\s	Пропуск (пробел, знак табуляции, символ новой строки и т.д.)
\S	Символ, не являющийся пропуском
\d	Цифра (то же, что [0-9])

\D	Не-цифра
\h	Шестнадцатеричная цифра (то же, что [0-9a-f])
\H	Нешестнадцатеричная цифра
\A	Начало строки символов (string)
\Z	Конец строки символов или позиция перед конечным символом новой строки
\z	Конец строки символов (string)
\b	Граница слова (только вне [])
\B	Не граница слова
\b	Забой (только внутри [])
[]	Произвольный набор символов
*	0 или более повторений предыдущего подвыражения
*?	0 или более повторений предыдущего подвыражения (нежадный алгоритм)
+	1 или более повторений предыдущего подвыражения
+?	1 или более повторений предыдущего подвыражения (нежадный алгоритм)
{m,n}	От m до n вхождений предыдущего подвыражения
{m,n}?	От m до n вхождений предыдущего подвыражения (нежадный алгоритм)
?	0 или 1 повторений предыдущего подвыражения
	Альтернативы
(?=)	Позитивное заглядывание вперед
(?!)	Негативное заглядывание вперед
()	Группировка подвыражений
(?>)	Вложенное подвыражение
(?:)	Несохраняющая группировка подвыражений
(?imx-imx)	Включить/выключить режимы, начиная с этого места
(?imx-imx:expr)	Включить/выключить режимы для этого выражения
(?#)	Комментарий

Регулярные выражения

```
reg1 = /[a-z&&[^aeiou]]/ # любая буква, кроме гласных а, е, и, о, у  
reg2 = /[a-z&&[^m-p]]/   # весь алфавит, кроме букв от m до p
```

```
pattern = /ax?b/  
pat2     = /a[xy]?b/  
pattern =~ "ab"      # 0  
pattern =~ "acb"     # nil  
pattern =~ "axb"     # 0  
pat2     =~ "ayb"     # 0  
pat2     =~ "acb"     # nil
```

```
pattern = /[0-9]+/  
pattern =~ "1"      # 0  
pattern =~ "2345678" # 0
```

```
str = "Where the sea meets the moon-blanch'd land,"  
match = /.the/.match(str)  
p match[0] # Вывести полученное соответствие:  
           # "Where the sea meets the"
```

```
str = "Where the sea meets the moon-blanch'd land,"  
match = /.?the/.match(str)  
p match[0] # Вывести полученное соответствие:  
           # "Where the"
```

Регулярные выражения

```
addresses =  
[ "409 W Jackson Ave",      "No. 27 Grande Place",  
  "16000 Pennsylvania Avenue", "2367 St. George St.",  
  "22 Rue Morgue",         "33 Rue St. Denis",  
  "44 Rue Zeeday",         "55 Santa Monica Blvd.",  
  "123 Main St., Apt. 234",  "123 Main St., #234",  
  "345 Euneva Avenue, Suite 23", "678 Euneva Ave, Suite A"]
```

```
regex = / ^                # Начало строки  
  ((No\.\?)\s+)?           # Необязательно: No[.]  
  \d+ \s+                  # Цифры и пробелы  
  ((\w|['-])+              # Название улицы... может  
  \s*                      # состоять из нескольких слов.  
)+  
(,\s*                     # Необязательно: запятая и т.д.  
  (Apt\.\?|Suite|\#)      # Apt[.], Suite, #  
  \s+                      # Пробелы  
  (\d+|[A-Z]))            # Цифры или одна буква  
)?  
$                          # Конец строки  
/x
```

```
num = "(\\d|[01]?\\d\\d{2[0-4]}\\d{25[0-5]})"
```

```
pat = "^({num}\\.){3}#{num}$"
```

```
ip_pat = Regexp.new(pat)
```

```
ip1 = "9.53.97.102"
```

```
if ip1 =~ ip_pat           # Печатается "да"  
  puts "да"  
else  
  puts "нет"  
end
```